ALGOBENCH

Benchmarking System Performance with Algorithms

A portfolio case study on combining Python desktop UI, DAA workloads, live system monitoring, SQLite history and Windows packaging into one focused benchmarking application.

Python	CustomTkinter	SQLite	Matplotlib	psutil	Git

Tushar Thakur

MCA Data Science · Chandigarh University

Project Type DAA / Python Mini Project	**Interface** Desktop Application	**Database** SQLite	**Launch** BAT / Windows EXE
Modules 8 Main Pages	**Benchmark** CPU / Memory / Algorithms / Stress	**Score** Weighted Overall Benchmark	**Theme** Light / Dark / System

PROJECT STORY

Why I Built AlgoBench

I wanted to build a DAA project that felt useful beyond a console demonstration. AlgoBench turns algorithm workloads into a desktop benchmarking experience where a user can monitor the machine, run CPU and memory tests, compare sorting methods and review an overall system score.

The project became an exercise in measurement as much as algorithm design. Workloads need to be repeatable, long-running tests must not freeze the interface, results must be understandable to normal users, and every completed benchmark should remain available through local history.

CONTEXT

The Problem

Algorithm projects often stop at console output
Sorting implementations can demonstrate theory, but they do not always show how workload size, system resources and real execution behavior connect.

What AlgoBench does
Turns DAA workloads into a visual desktop benchmark with live monitoring, controlled tests, history and an overall score.

KEY CAPABILITIES

What the Application Can Do

CPU & memory
Run Light, Medium and Heavy workloads and measure execution time, throughput, CPU activity and memory behavior.

Algorithms & comparison
Compare Bubble, Insertion, Merge and Quick Sort on the same generated dataset with execution-time bars.

Stress & complete test
Apply sustained controlled load, monitor stability, or run CPU, memory, processing and stress checks as one sequence.

Dashboard & system overview
Track live CPU, memory, storage and battery information while viewing processor, cores, threads and uptime.

History & settings
Store completed benchmark results locally, clear or refresh history, and persist appearance and dashboard refresh settings.

Offline delivery
Run directly with Python, launch through a BAT file, or package the application as a Windows EXE using PyInstaller.

ARCHITECTURE

How AlgoBench Is Structured

AlgoBench follows a compact offline desktop architecture: CustomTkinter provides the interface, Python benchmark modules coordinate workloads, algorithm and monitoring helpers produce measurements, SQLite stores results and settings, and Matplotlib turns live system activity into visual feedback.

UI → **Tests** → **Engine** → **SQLite** → **Results**

Layer	Main Technology	Responsibility
Interface	CustomTkinter	Desktop pages, reusable cards, navigation, themes, progress indicators and result presentation.
Benchmark	Python + sorting algorithms	Workload generation, timing, scoring, algorithm comparison, stress execution and overall-score calculation.
Data & insight	psutil + SQLite + Matplotlib	System metrics, persistent history/settings and live performance visualization.

EXAMPLE WORKFLOW

Data Flow: Running the Complete System Test

Start → CPU → Memory → Algorithms → Stress → Score → Save

The Complete System Test runs the main benchmark modules in sequence. CPU and memory workloads are measured first, sorting methods are compared next, and a controlled stress test checks stability. The resulting scores are combined with fixed weights, compared with the project reference target and stored locally for later review.

CORE RELATIONSHIPS

Local Data Model

Entity	Connection	Usage
Benchmark result	Test type / mode	History, score and result details
Application setting	Key / value	Appearance and dashboard refresh rate
System metric	CPU / RAM / battery / storage	Dashboard and System Overview

LOCAL DATA ENGINEERING

The Data Work Behind the UI

AlgoBench keeps persistence intentionally simple. SQLite stores benchmark history and application settings, while small database helpers isolate storage logic from the interface. Monitoring helpers collect CPU, memory, storage, battery and processor information so the UI can focus on presentation.

Object	Examples	Purpose
SQLite table	test_history	Persist completed benchmark scores, ratings, modes and detailed result data.
Settings table	app_settings	Store appearance mode and dashboard refresh preference between launches.
Database helpers	save / fetch / clear history	Keep SQL operations reusable and separated from screen layout code.
Monitoring helpers	CPU, memory, disk, battery, uptime	Provide one reusable source of live system information for multiple pages.

RELIABILITY

Responsive Tests & Safe State

BACKGROUND THREADS
Keep long benchmarks off the UI thread

SAFE UI UPDATES
Return worker results to the main interface

LOCAL DATABASE
Keep history and settings on-device

CONTROLLED STOP
Allow stress testing to end safely

ENGINEERING CHOICES

A Few Design Decisions I’m Proud Of

DAA in the backend, simple language in the UI

Bubble Sort, Insertion Sort, Merge Sort and Quick Sort remain explicit in the implementation, while the interface presents friendly processing names so the application works for both demonstration and normal use.

Controlled workload tiers

Light, Medium and Heavy modes scale the amount of work instead of changing the purpose of the test, making benchmark behavior easier to explain and repeat.

Explainable complete-system score

The overall score combines CPU performance at 35%, memory at 30%, processing performance at 20% and stress stability at 15%, so the final result is tied to visible benchmark components.

Offline desktop delivery

The project runs without a web server or external database. A BAT launcher simplifies demonstrations, while PyInstaller can package AlgoBench as a Windows executable.

SCOPE

What the Project Includes - and What It Does Not

Included

✓ Live CPU, memory, storage and battery dashboard

✓ Detailed system and processor overview

✓ CPU benchmark with Light / Medium / Heavy workloads

✓ Memory benchmark with allocation tracking

✓ Bubble, Insertion, Merge and Quick Sort comparison

✓ Controlled stress testing with live monitoring

✓ Complete weighted system score and rating

✓ Internal reference-score comparison

Intentionally out of scope

– GPU benchmarking

– Network benchmark

– Dedicated disk read / write benchmark

– Cloud synchronization

– User accounts or online authentication

– Official hardware certification or global ranking

– Online benchmark-result database

– Mobile or web companion application

REFLECTION

What I Learned

AlgoBench helped me connect algorithm theory with practical software engineering. The most useful lessons came from measuring real execution, keeping the GUI responsive during heavy work, separating monitoring and database logic from presentation, and turning raw benchmark data into results a user can understand.

Desktop UI architecture **Reusable pages, components, cards, responsive layouts and navigation.**	**DAA benchmarking** **Sorting algorithms, workload scaling, execution timing and practical comparison.**	**System monitoring** **CPU, memory, storage, battery, processor and uptime information.**
Data persistence **SQLite history, JSON-style result details and persistent application settings.**	**Concurrency & UX** **Background tests, safe UI updates, progress reporting and controlled stress-stop behavior.**	**Project delivery** **Git, GitHub README, documentation, BAT launcher, PyInstaller packaging and portfolio presentation.**

CONCLUSION

Final Thoughts

AlgoBench began as a DAA mini-project, but it became a practical desktop benchmarking application. It combines sorting algorithms, workload scaling, system monitoring, SQLite persistence, multithreading, charts and local configuration without depending on a browser, cloud service or database server.

For me, the strongest part of the project is the connection between theory and experience: algorithms create measurable workloads, system metrics explain what the machine is doing, history preserves results, and the Complete System Test turns multiple measurements into one understandable summary. That flow makes AlgoBench feel like a product rather than a collection of algorithm demos.

EXPLORE

Project Links

GitHub	https://github.com/tushar-011
LinkedIn	https://www.linkedin.com/in/tushar-thakur-8848a7396/
Email	artificial.thakur@gmail.com